The Spring Framework was first developed by Rod Johnson in 2003. It was developed to make the development of Java applications quicker, easier and safer for developers. Spring Framework is an open-source, lightweight, easy-to-build framework that can also be considered as a framework of frameworks as it houses various frameworks like Dependency Injection, Spring MVC, Spring JDBC, Spring Hibernate, Spring AOP, EJB, JSF, etc. A key element of Spring is the feature of the application's infrastructural support. Spring focuses on providing the "plumbing" of enterprise-level applications and ensures that the developers can focus purely on application-level business logic, without unnecessary ties to specific deployment environments. Applications developed in Spring are more reliable, scalable, and very simple to build and maintain. Spring was developed as means to help developers manage the business objects of the application. Due to its vast features and flexibility, Spring became the most loved framework for developing enterprise-level Java-based applications.

In the following section, we will see what are the most commonly asked interview questions and answers to prepare you for Spring-based interviews.

Spring, Spring Core, Spring IoC Interview Questions

1. What are the bean scopes available in Spring?

The Spring Framework has five scope supports. They are:

- Singleton: The scope of bean definition while using this would be a single instance per IoC container.
- Prototype: Here, the scope for a single bean definition can be any number of object instances.
- Request: The scope of the bean definition is an HTTP request.
- Session: Here, the scope of the bean definition is HTTP-session.
- Global-session: The scope of the bean definition here is a Global HTTP session.

Note: The last three scopes are available only if the users use web-aware ApplicationContext containers.

2. What are the limitations of autowiring?

- Overriding possibility: Dependencies are specified using <constructor-arg> and <property> settings that override autowiring.
- Data types restriction: Primitive data types, Strings, and Classes can't be autowired.

3. What is autowiring and name the different modes of it?

The IoC container autowires relationships between the application beans. Spring lets collaborators resolve which bean has to be wired automatically by inspecting the contents of the BeanFactory.
Different modes of this process are:

- no: This means no autowiring and is the default setting. An explicit bean reference should be used for wiring.
- byName: The bean dependency is injected according to the name of the bean. This matches and wires its properties with the beans defined by the same names as per the configuration.
- byType: This injects the bean dependency based on type.
- constructor: Here, it injects the bean dependency by calling the constructor of the class. It has a large number of parameters.
- autodetect: First the container tries to wire using autowire by the constructor, if it isn't possible then it tries to autowire by byType.

4. What do you understand by Bean Wiring.

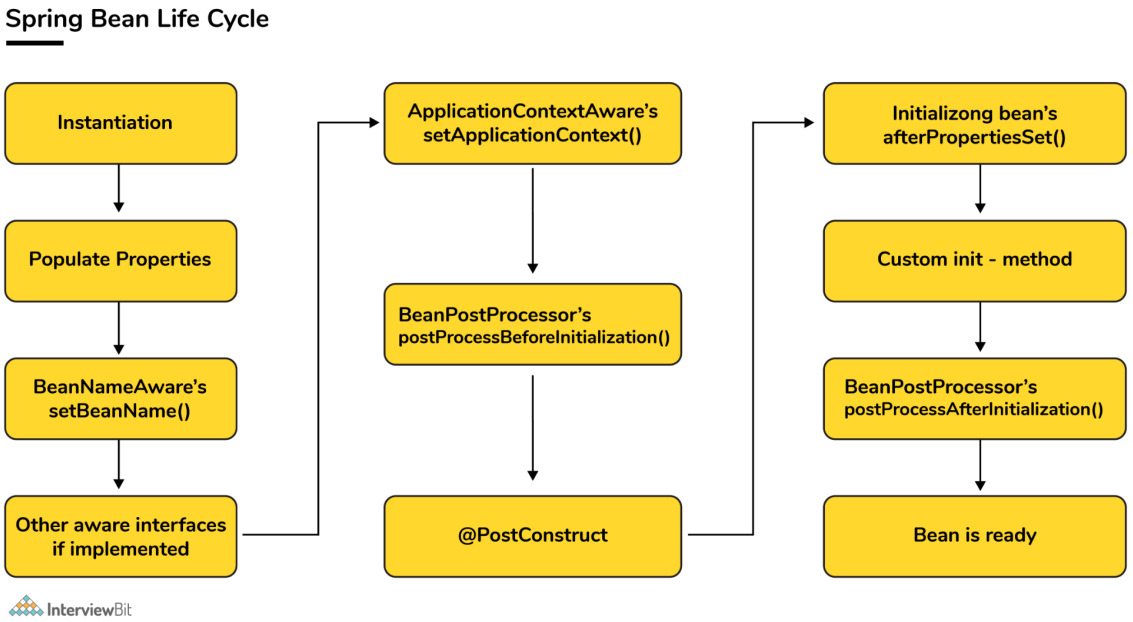
- When beans are combined together within the Spring container, they are said to be wired or the phenomenon is called bean wiring.
- The Spring container should know what beans are needed and how the beans are dependent on each other while wiring beans. This is given by means of XML / Annotations / Java code-based configuration.

5. Explain Bean life cycle in Spring Bean Factory Container.

The Bean life cycle is as follows:

- The IoC container instantiates the bean from the bean's definition in the XML file.

- Spring then populates all of the properties using the dependency injection as specified in the bean definition.
- The bean factory container calls setBeanName() which take the bean ID and the corresponding bean has to implement BeanNameAware interface.
- The factory then calls setBeanFactory() by passing an instance of itself (if BeanFactoryAware interface is implemented in the bean).
- If BeanPostProcessors is associated with a bean, then the preProcessBeforeInitialization() methods are invoked.
- If an init-method is specified, then it will be called.
- Lastly, postProcessAfterInitialization() methods will be called if there are any BeanPostProcessors associated with the bean that needs to be run post creation.



6. How is the configuration meta data provided to the spring container?

There are 3 ways of providing the configuration metadata. They are as follows:

- XML-Based configuration: The bean configurations and their dependencies are specified in XML configuration files. This starts with a bean tag as shown below:

```
<bean id="interviewBitBean" class="org.intervuewBit.firstSpring.InterviewBitBean">
  <property name="name" value="InterviewBit"></property>
```

```
  </bean>
```

- Annotation-Based configuration: Instead of the XML approach, the beans can be configured into the component class itself by using annotations on the relevant class, method, or field declaration.
    - Annotation wiring is not active in the Spring container by default. This has to be enabled in the Spring XML configuration file as shown below

```
<beans>
<context:annotation-config/>
<!-- bean definitions go here -->
</beans>
```

- Java-based configuration: Spring Framework introduced key features as part of new Java configuration support. This makes use of the @Configuration annotated classes and @Bean annotated methods. Note that:
    - @Bean annotation has the same role as the <bean/> element.
    - Classes annotated with @Configuration allow to define inter-bean dependencies by simply calling other @Bean methods in the same class.

7. What are Spring Beans?

- They are the objects forming the backbone of the user's application and are managed by the Spring IoC container.
- Spring beans are instantiated, configured, wired, and managed by IoC container.
- Beans are created with the configuration metadata that the users supply to the container (by means of XML or java annotations configurations.)

Check out Scaler Topics' Free Java Spring Boot course to master the fundamentals of Java Spring Boot.

8. Explain the difference between constructor and setter injection?

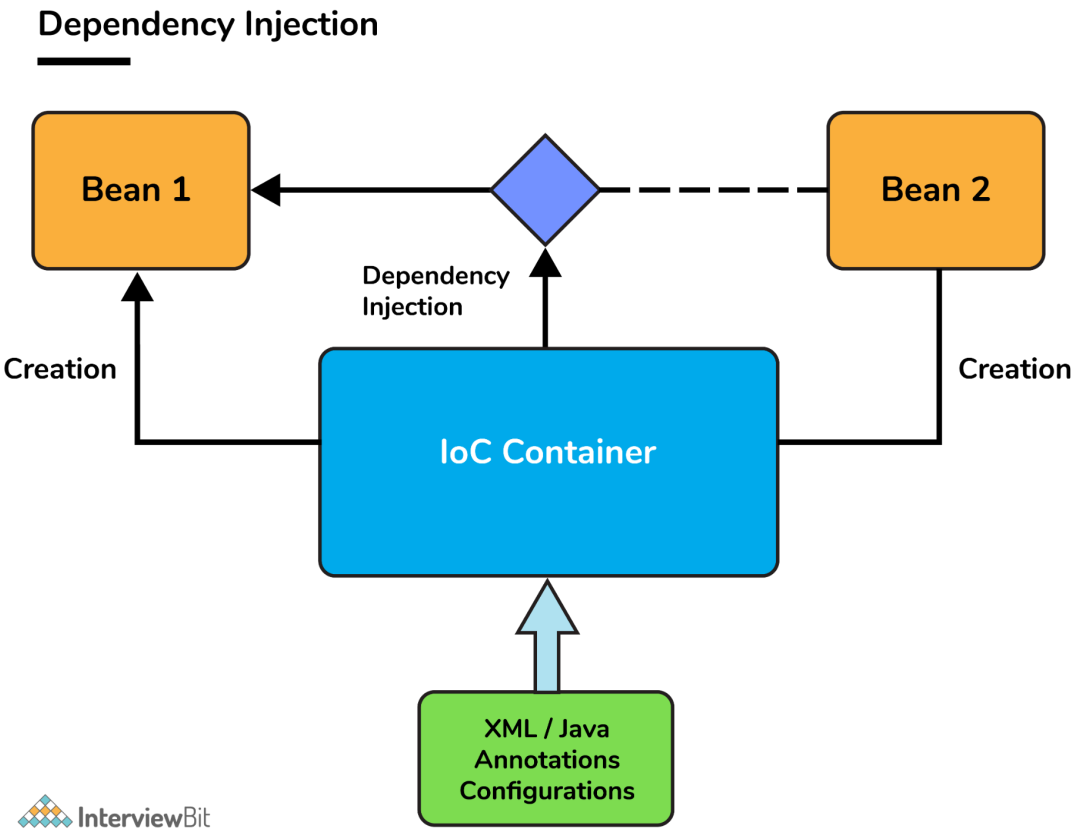
- In constructor injection, partial injection is not allowed whereas it is allowed in setter injection.
- The constructor injection doesn't override the setter property whereas the same is not true for setter injection.

- Constructor injection creates a new instance if any modification is done. The creation of a new instance is not possible in setter injection.
- In case the bean has many properties, then constructor injection is preferred. If it has few properties, then setter injection is preferred.

9. What do you understand by Dependency Injection?

The main idea in Dependency Injection is that you don't have to create your objects but you just have to describe how they should be created.

- The components and services need not be connected by us in the code directly. We have to describe which services are needed by which components in the configuration file. The IoC container present in Spring will wire them up together.

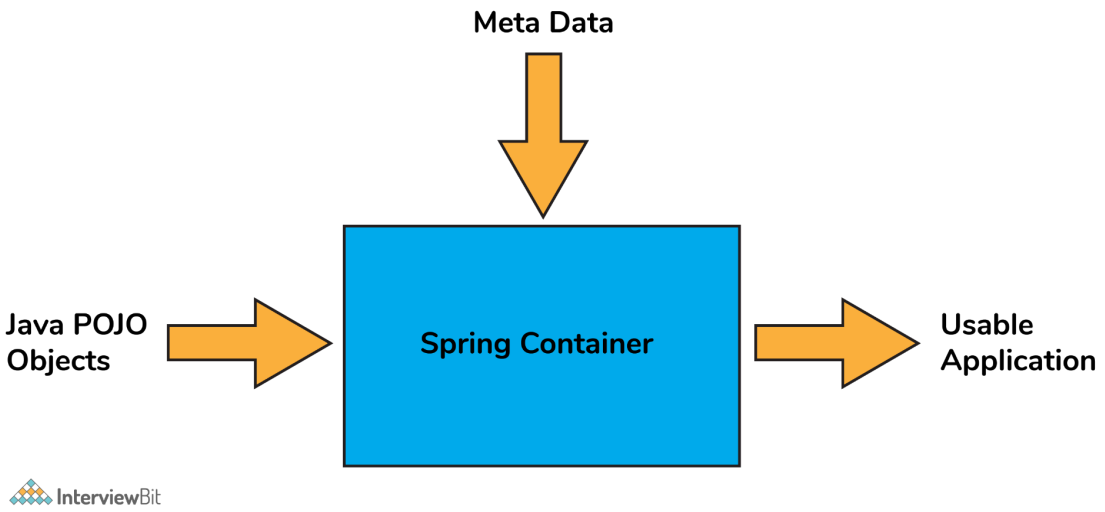


- In Java, the 2 major ways of achieving dependency injection are:
  - Constructor injection: Here, the IoC container invokes the class constructor with a number of arguments where each argument represents a dependency on the other class.

- Setter injection: Here, the spring container calls the setter methods on the beans after invoking a no-argument static factory method or default constructor to instantiate the bean.

10. What do you mean by IoC (Inversion of Control) Container?

Spring container forms the core of the Spring Framework. The Spring container uses Dependency Injection (DI) for managing the application components by creating objects, wiring them together along with configuring and managing their overall life cycles. The instructions for the spring container to do the tasks can be provided either by XML configuration, Java annotations, or Java code.



11. What is a Spring configuration file?

A Spring configuration file is basically an XML file that mainly contains the classes information and describes how those classes are configured and linked to each other. The XML configuration files are verbose and cleaner.

12. What are the features of Spring Framework?

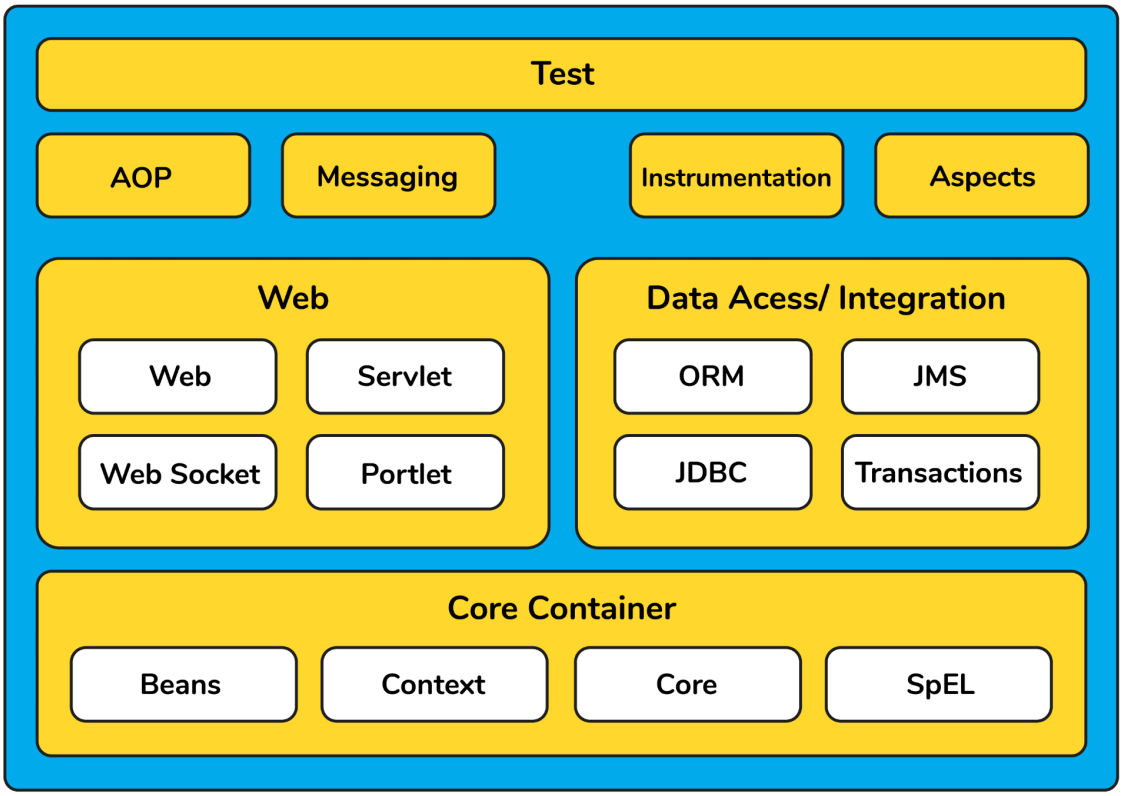
- Spring framework follows layered architecture pattern that helps in the necessary components selection along with providing a robust and cohesive framework for J2EE applications development.
- The AOP (Aspect Oriented Programming) part of Spring supports unified development by ensuring separation of application's business logic from other system services.

- Spring provides highly configurable MVC web application framework which has the ability to switch to other frameworks easily.
- Provides provision of creation and management of the configurations and defining the lifecycle of application objects.
- Spring has a special design principle which is known as IoC (Inversion of Control) that supports objects to give their dependencies rather than looking for creating dependent objects.
- Spring is a lightweight, java based, loosely coupled framework.
- Spring provides generic abstraction layer for transaction management that is also very useful for container-less environments.
- Spring provides a convenient API to translate technology-specific exceptions (thrown by JDBC, Hibernate or other frameworks) into consistent, unchecked exceptions. This introduces abstraction and greatly simplifies exception handling.

13. What is Spring Framework?

- Spring is a powerful open-source, loosely coupled, lightweight, java framework meant for reducing the complexity of developing enterprise-level applications. This framework is also called the “framework of frameworks” as spring provides support to various other important frameworks like JSF, Hibernate, Structs, EJB, etc.
- There are around 20 modules which are generalized into the following types:
    - Core Container
    - Data Access/Integration
    - Web
    - AOP (Aspect Oriented Programming)
    - Instrumentation
    - Messaging
    - Test

Spring Framework



InterviewBit